

INFOM – Information Management

FINAL PROJECT: Sales Database System

Gian Paris Agsam,

BSIT 1C BSIT,

College of Computing

Studies

April 24, 2026

Project Title: Sales Database System

Project Overview

You already created the core tables of a sales database:

- Customers
- Products
- Users
- Sales

For this final project, you will **improve and extend** your database by adding new tables that make the system more realistic and complete.

You will add:

1. **Orders** – stores customer orders
2. **Order_Items** – stores the products included in each order
3. **Logs** – stores user activity logs

These new tables will help transform your database from a simple record system into a more complete **transaction-based information system**.

Objective

At the end of this final project, the students should be able to:

1. extend an existing relational database;
2. create additional tables with proper relationships;
3. apply primary keys and foreign keys correctly;
4. insert valid data into related tables;
5. use joins to display related records;
6. understand the purpose of transaction tables and activity logs;
7. present a working database design for a sales system.

Introduction

A database system is a structured way of storing, managing, and retrieving information efficiently. In real-world applications such as business and sales environments, databases are used to handle large amounts of data involving customers, products, transactions, and system activities.

This project, titled **Sales Database System**, builds upon an existing relational database that already includes core tables such as Customers, Products, Users, and Sales. While these tables provide a foundation for storing basic business data, a more realistic sales system requires additional structures to properly represent transactions and system behavior.

To improve the functionality and realism of the system, three new tables are introduced:

- **Orders** – stores overall customer purchase transactions
- **Order_Items** – stores detailed product information within each order
- **Logs** – records user activities within the system for tracking and security purposes

These additional tables enhance the database by introducing proper relational links, allowing one-to-many relationships between customers and orders, and between orders and products. The Logs table further improves system accountability by tracking user actions such as login, order creation, and updates.

Through this extension, the database evolves from a simple data storage system into a more complete transaction-based information system. It demonstrates the importance of relational database design principles such as normalization, foreign key relationships, indexing, and data integrity.

This project also provides practical experience in implementing SQL operations including table creation, data insertion, joins, and query testing, which are essential skills in developing real-world database-driven applications.

Database Tables Added

- **Orders** – stores overall customer purchase transactions
- **Order_Items** – stores detailed product information within each order
- **Logs** – records user activities within the system for tracking and security purposes

SQL implementation

1. Student Task

You are required to **add three new tables** to your existing database:

- Orders
- Order_Items
- Logs

Then you must:

1. create the tables;
2. add the necessary foreign keys;
3. insert sample values;
4. test the tables using SQL queries;
5. explain the role of each table in your final report or presentation.

1. Tables to Add

2. Orders

Stores the main order information of each customer.

2. Order_Items

Stores the products included in each order.

3. Logs

Stores system activity done by users.

Complete SQL Script

Use this script after your existing tables are already created.

```
USE inv;

-- =====
-- ORDERS TABLE
-- =====
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATE NOT NULL,
    total DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

-- =====
-- ORDER_ITEMS TABLE
-- =====
CREATE TABLE Order_Items (
    order_item_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

-- =====
-- LOGS TABLE
-- =====
CREATE TABLE Logs (
    log_id INT PRIMARY KEY,
    user_id INT NOT NULL,
    action VARCHAR(100) NOT NULL,
    log_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

```

-- =====
-- INDEXES
-- =====

CREATE INDEX idx_orders_customer_id ON Orders(customer_id);
CREATE INDEX idx_orders_order_date ON Orders(order_date);

CREATE INDEX idx_order_items_order_id ON Order_Items(order_id);
CREATE INDEX idx_order_items_product_id ON Order_Items(product_id);

CREATE INDEX idx_logs_user_id ON Logs(user_id);
CREATE INDEX idx_logs_log_time ON Logs(log_time);

-- =====
-- SAMPLE DATA FOR ORDERS
-- =====
INSERT INTO Orders (order_id, customer_id, order_date, total)
VALUES
(2001, 1, '2024-04-01', 1200.00),
(2002, 2, '2024-04-02', 300.00),
(2003, 3, '2024-04-03', 800.00);

-- =====
-- SAMPLE DATA FOR ORDER_ITEMS
-- =====
INSERT INTO Order_Items (order_item_id, order_id, product_id,
quantity, price)
VALUES
(1, 2001, 101, 1, 1200.00),
(2, 2002, 103, 2, 150.00),
(3, 2003, 102, 1, 800.00);

-- =====
-- SAMPLE DATA FOR LOGS
-- =====
INSERT INTO Logs (log_id, user_id, action)
VALUES
(1, 1, 'User Login'),
(2, 1, 'Created Order 2001'),

```

```
(3, 2, 'Created Order 2002'),  
(4, 1, 'Viewed Sales Report'),  
(5, 3, 'Updated Product Stock');
```

Explanation of Each Table

1. Orders

This table stores the main details of an order.

Fields:

- order_id → unique order number
- customer_id → identifies which customer made the order
- order_date → date of the order
- total → total amount of the order

Purpose:

This table connects customers to their purchases.

2. Order_Items

This table stores the products included in each order.

Fields:

- order_item_id → unique order item number
- order_id → identifies which order the item belongs to
- product_id → identifies the product
- quantity → number of items ordered
- price → price per item or recorded price for that order item

Purpose:

This allows one order to contain multiple products.

Example:

- Order 2001 may contain 1 laptop
- Order 2002 may contain 2 headphones

3. Logs

This table stores user actions inside the system.

Fields:

- log_id → unique log number

- user_id → identifies which user performed the action
- action → description of the activity
- log_time → date and time of the action

Purpose:

This is useful for system tracking, security, and monitoring.

Example actions:

- login
- create order
- update product
- delete sale

Required SQL Tests

After creating the tables and inserting the data, test the following queries.

1. View all orders

```
SELECT * FROM Orders;
```

1. View all order items

```
SELECT * FROM Order_Items;
```

1. View all logs

```
SELECT * FROM Logs;
```

1. Show customers and their orders

```
SELECT
    o.order_id,
    c.first_name,
    c.last_name,
    o.order_date,
    o.total
FROM Orders o
INNER JOIN Customers c ON o.customer_id = c.customer_id;
```

1. Show order details with products

```
SELECT
    oi.order_item_id,
    oi.order_id,
```

```

        p.product_name,
        oi.quantity,
        oi.price
FROM Order_Items oi
INNER JOIN Products p ON oi.product_id = p.product_id;

```

1. Show logs with usernames

```

SELECT
    l.log_id,
    u.username,
    l.action,
    l.log_time
FROM Logs l
INNER JOIN Users u ON l.user_id = u.user_id;

```

Sample outputs

Server: 127.0.0.1 » Database: im_security_lab » Table: orders

[Browse](#)
[Structure](#)
[SQL](#)
[Search](#)
[Insert](#)
[Export](#)
[Import](#)
[Privileges](#)
[Operations](#)
[Tracking](#)
[Triggers](#)

[Table structure](#)
[Relation view](#)

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	order_id	int(11)			No	None			Change Drop More
☐ 2	customer_id	int(11)			No	None			Change Drop More
☐ 3	order_date	date			No	None			Change Drop More
☐ 4	total	decimal(10,2)			No	None			Change Drop More

```

-- =====
-- ORDERS TABLE
-- =====

CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATE NOT NULL,
    total DECIMAL(10,2) NOT NULL,

```

```
FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 order_item_id	int(11)			No	None			Change Drop More
☐	2 order_id	int(11)			No	None			Change Drop More
☐	3 product_id	int(11)			No	None			Change Drop More
☐	4 quantity	int(11)			No	None			Change Drop More
☐	5 price	decimal(10,2)			No	None			Change Drop More

```
-- =====
-- ORDER_ITEMS TABLE
-- =====

CREATE TABLE Order_Items (
    order_item_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 log_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 user_id	int(11)			No	None			Change Drop More
☐	3 action	varchar(100)	latin1_swedish_ci		No	None			Change Drop More
☐	4 log_time	datetime			Yes	current_timestamp()			Change Drop More

```
-- =====
-- LOGS TABLE
-- =====

CREATE TABLE Logs (
    log_id INT PRIMARY KEY,
    user_id INT NOT NULL,
    action VARCHAR(100) NOT NULL,
    log_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
orders	0	PRIMARY	1	order_id	A	3		NULL	NULL	BTREE		
orders	1	idx_orders_customer_id	1	customer_id	A	3		NULL	NULL	BTREE		
orders	1	idx_orders_order_date	1	order_date	A	3		NULL	NULL	BTREE		

```
-- =====
-- INDEXES
-- =====

CREATE INDEX idx_orders_customer_id ON Orders(customer_id);
CREATE INDEX idx_orders_order_date ON Orders(order_date);

CREATE INDEX idx_order_items_order_id ON Order_Items(order_id);
CREATE INDEX idx_order_items_product_id ON Order_Items(product_id);

CREATE INDEX idx_logs_user_id ON Logs(user_id);
CREATE INDEX idx_logs_log_time ON Logs(log_time);
```

		order_id	customer_id	order_date	total
☐ Edit Copy Delete		2001	1	2024-04-01	1200.00
☐ Edit Copy Delete		2002	2	2024-04-02	300.00
☐ Edit Copy Delete		2003	3	2024-04-03	800.00

```
-- =====
-- SAMPLE DATA FOR ORDERS
-- =====
INSERT INTO Orders (order_id, customer_id, order_date, total)
VALUES
(2001, 1, '2024-04-01', 1200.00),
(2002, 2, '2024-04-02', 300.00),
(2003, 3, '2024-04-03', 800.00);
```

	order_item_id	order_id	product_id	quantity	price
☐  Edit  Copy  Delete	1	2001	1	1	1200.00
☐  Edit  Copy  Delete	2	2002	3	2	150.00
☐  Edit  Copy  Delete	3	2003	2	1	800.00

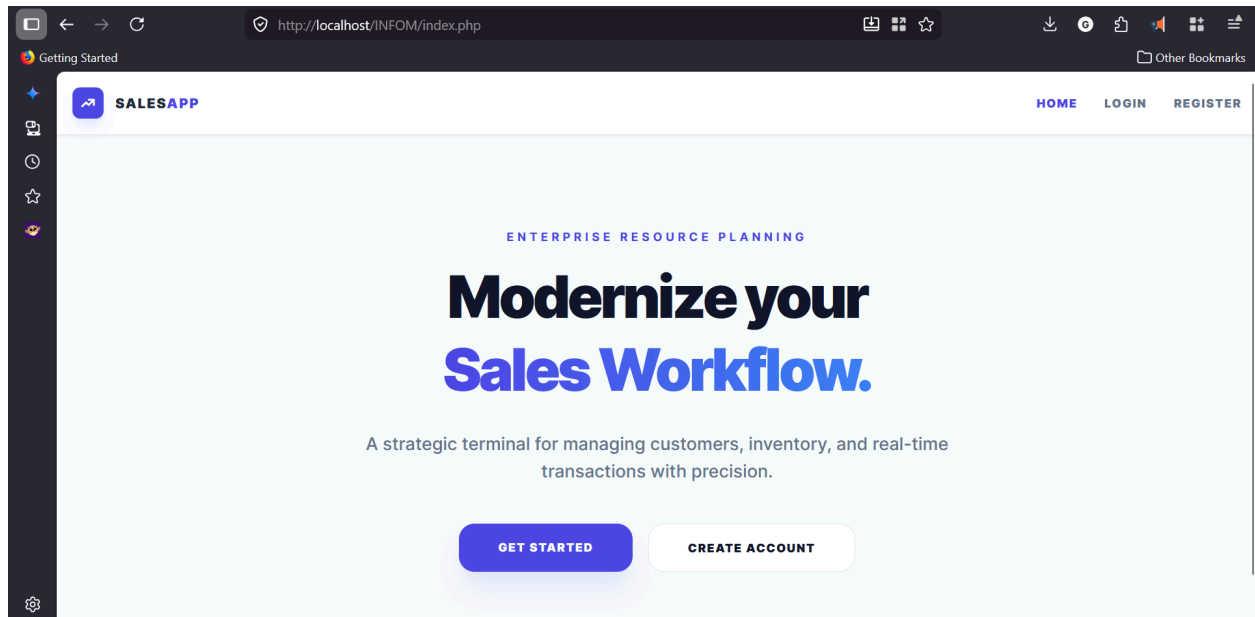
```
-- =====
-- SAMPLE DATA FOR ORDER_ITEMS
-- =====
INSERT INTO Order_Items (order_item_id, order_id, product_id,
quantity, price)
VALUES
(1, 2001, 101, 1, 1200.00),
(2, 2002, 103, 2, 150.00),
(3, 2003, 102, 1, 800.00);
```

				log_id	user_id	action	log_time
☐		Edit		Copy		Delete	
				1	5	User Login	2026-04-13 15:57:58
☐		Edit		Copy		Delete	
				2	5	Created Order 2001	2026-04-13 15:57:58
☐		Edit		Copy		Delete	
				3	3	Created Order 2002	2026-04-13 15:57:58
☐		Edit		Copy		Delete	
				4	5	Viewed Sales Report	2026-04-13 15:57:58
☐		Edit		Copy		Delete	
				5	3	Updated Product Stock	2026-04-13 15:57:58

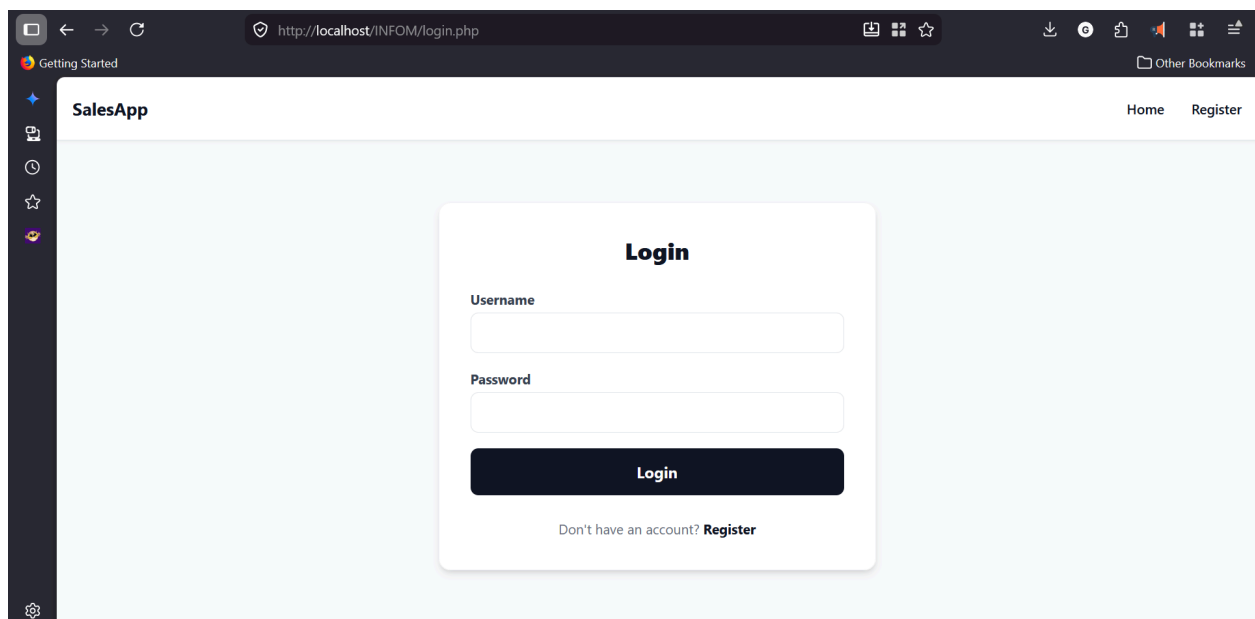
```
-- =====
-- SAMPLE DATA FOR LOGS
-- =====
INSERT INTO Logs (log_id, user_id, action)
VALUES
(1, 1, 'User Login'),
(2, 1, 'Created Order 2001'),
(3, 2, 'Created Order 2002'),
(4, 1, 'Viewed Sales Report'),
(5, 3, 'Updated Product Stock');
```

System Features

index.php(default page)



Login Page



Register page

Getting Started

Other Bookmarks

Home Login

Create Account

Username

Password

Role

User

Register

Already have an account? [Login](#)

Admin dashboard

Getting Started

Other Bookmarks

SALESAPP

WELCOME BACK, admin

System Status ONLINE & SECURED AD

MAIN MENU

- Dashboard
- Customers
- Products
- Sales

ADMINISTRATION

- Users
- System Logs

SIGN OUT

REVENUE

\$262

TOTAL ORDERS

3

CUSTOMERS

6

RECENT SALES

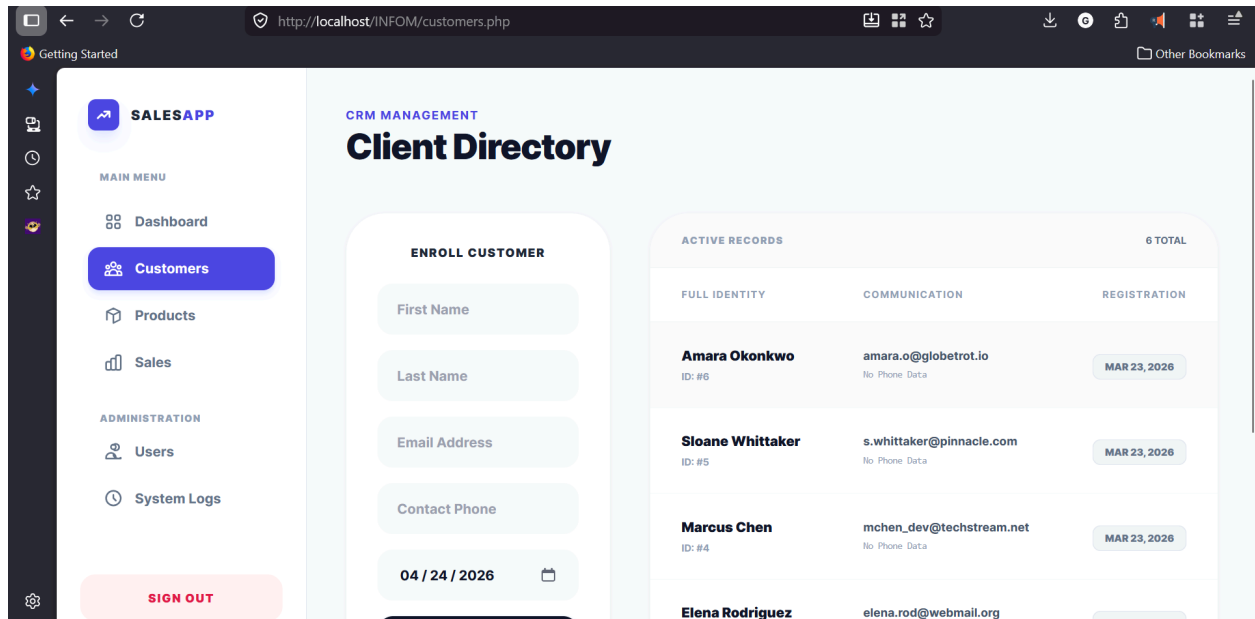
SEE ALL ACTIVITY

Amara Okonkwo APR 22	ALL-WEATHER VENTURE PARKA	\$85.50
Elena Rodriguez MAR 23	LUMINA PRO WIRELESS HEADPHONES	\$149.00

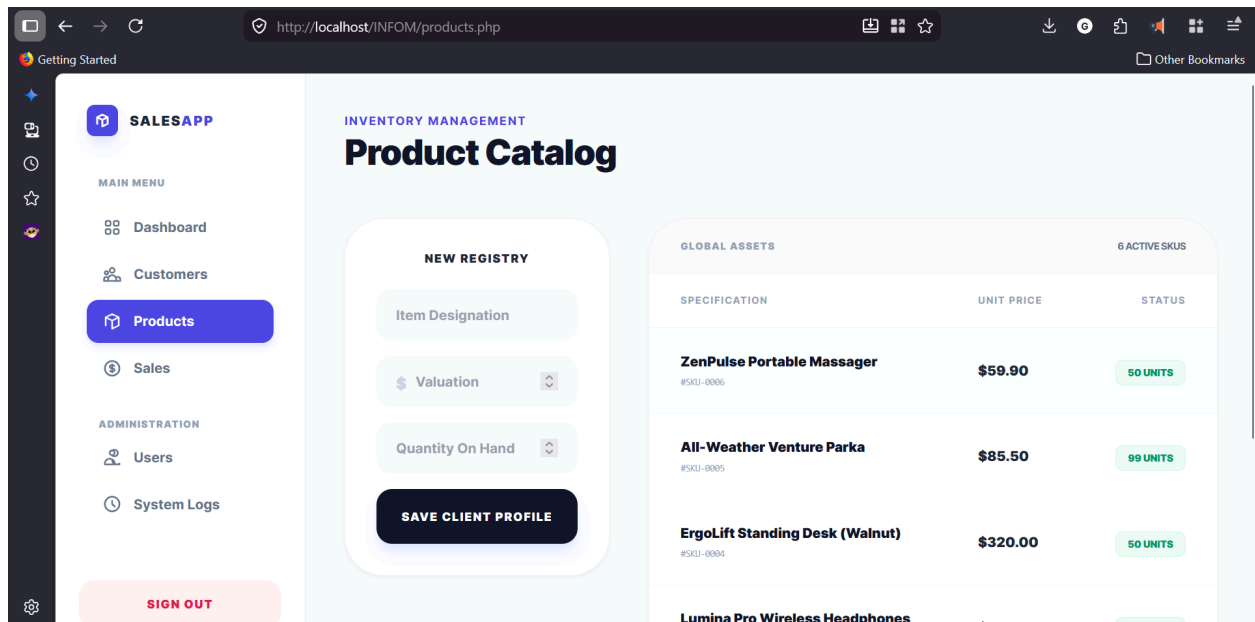
INVENTORY

PROCESS SALE

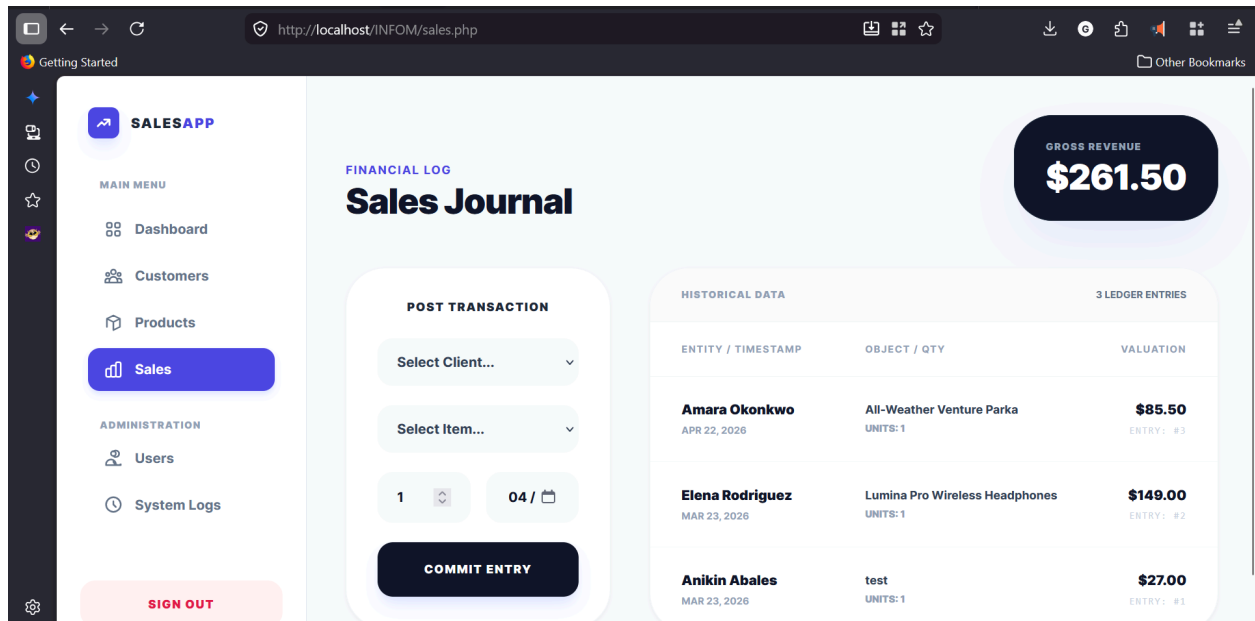
Customers page



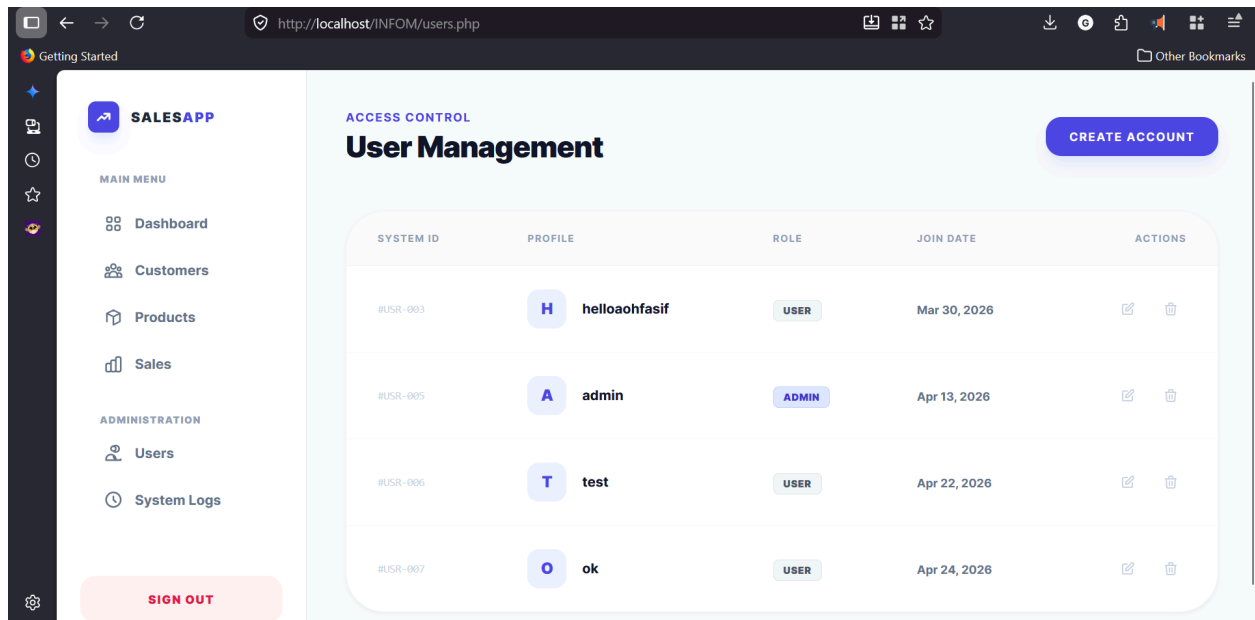
Products page



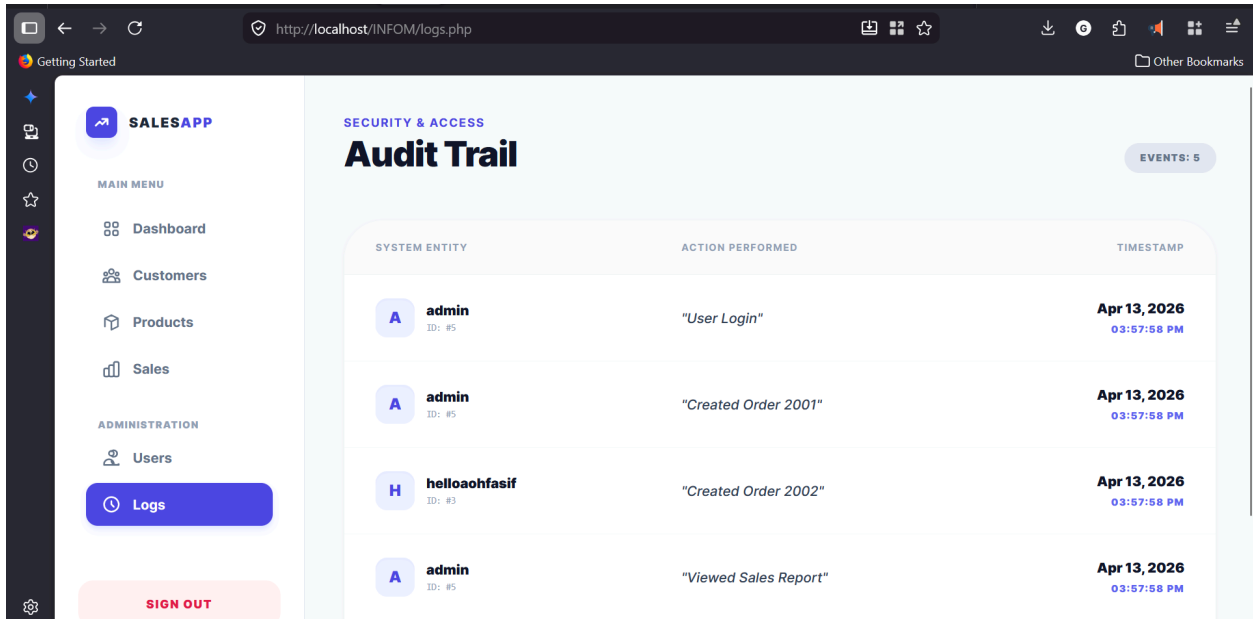
Sales page



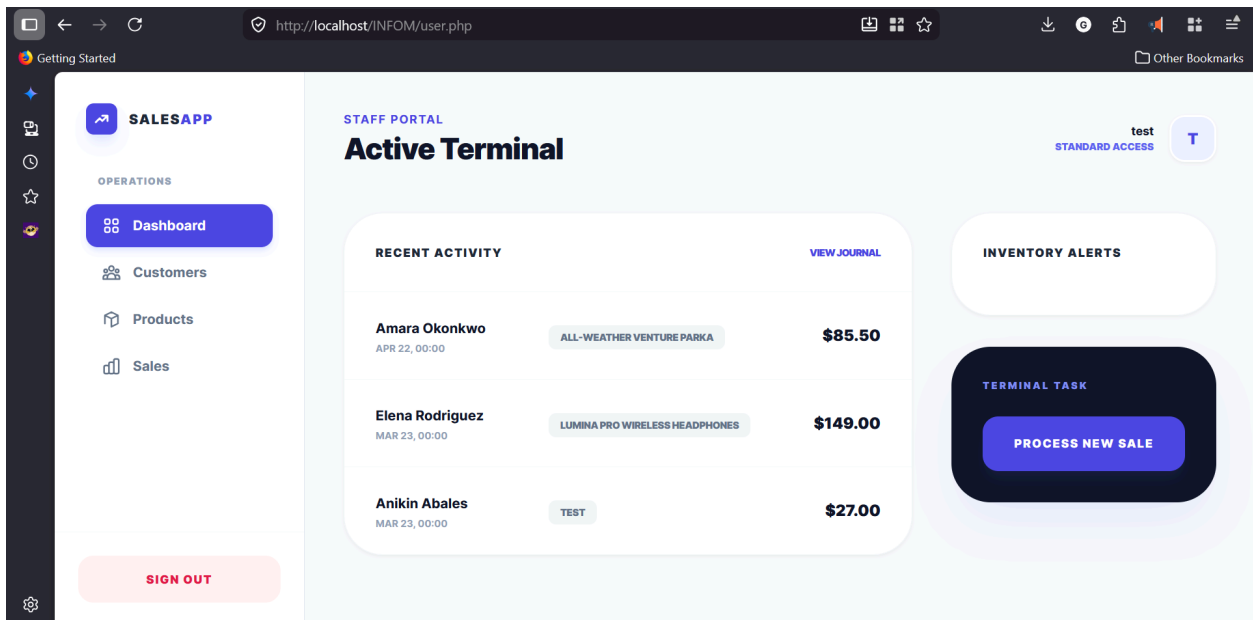
Users page(For admin role only)



Logs page(For admin role only)



User dashboard



Discussion

The Sales Database System was improved by adding Orders, Order_Items, and Logs tables. These tables make the database more realistic by supporting customer transactions, product details per order, and user activity tracking.

The Orders and Order_Items tables show proper relationships between customers and products, while the Logs table helps monitor user actions for security and system tracking. Overall, the system demonstrates the use of foreign keys, joins, and relational database design principles.

I have also successfully integrated the database with the GUI and successfully implemented a secure login system with RBAC([Role-Based Access Control](#)).

Conclusion

The project successfully enhanced the basic database into a functional sales system. The added tables improved data organization, relationships, and system tracking.

This project demonstrates practical skills in SQL, including table creation, relationships, and data handling, making the system more complete and realistic.

Expected Output / Deliverables

Each student or group must submit the following:

1. SQL File

A .sql file containing:

- table creation for Orders, Order_Items, and Logs
- indexes
- insert statements
- test queries

2. Screenshot Evidence

Screenshots showing:

- successful creation of tables

- successful insertion of data
- successful query outputs

3. **Final Project Report**

4. GUI CRUD

Sales Database System - Php Crud · php

System Features and Database components Rubric (50 Points)